

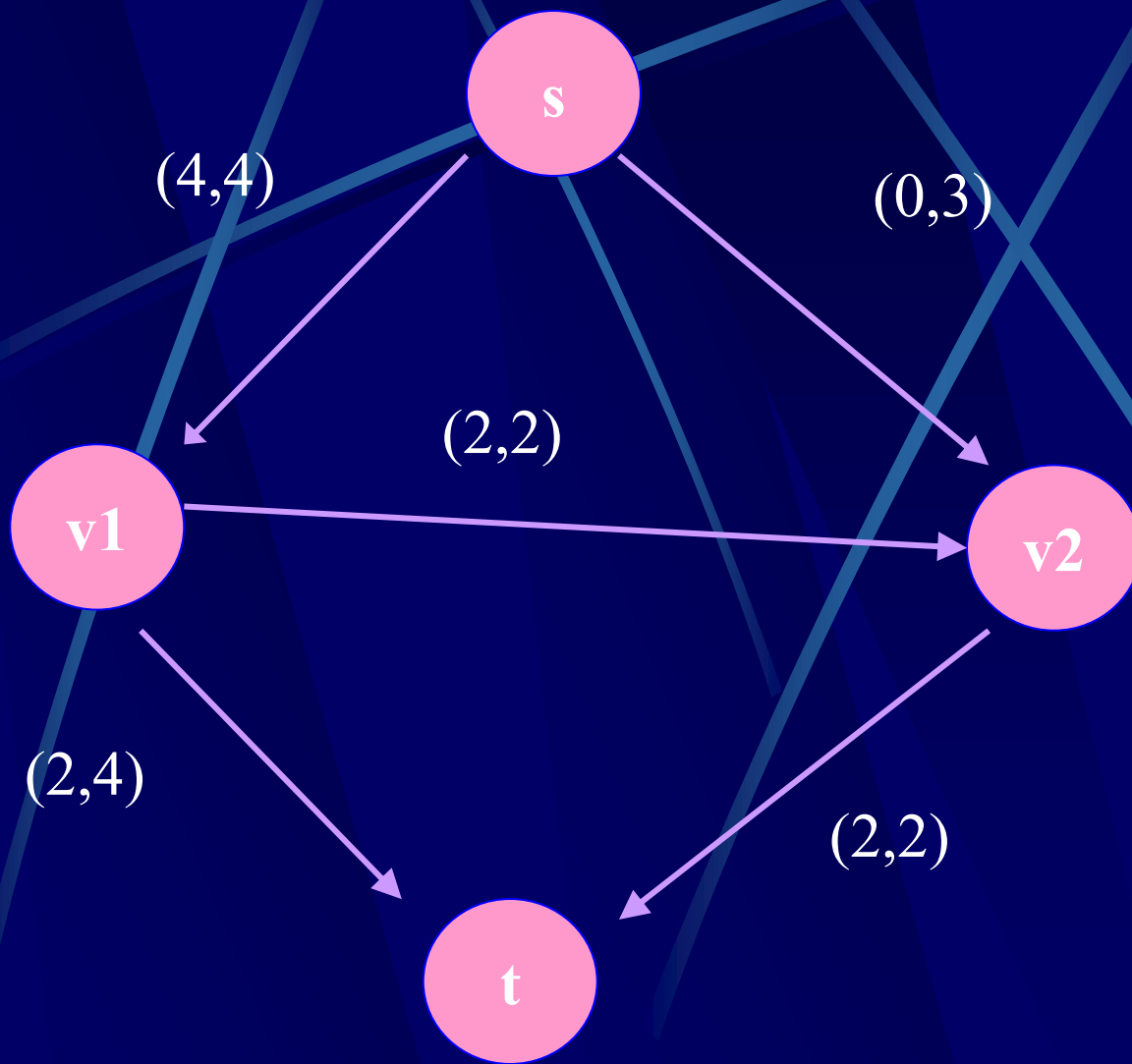


网络流

杭州学军中学 魏越闽

一些符号和定义

- V 表示整个图中的所有结点的集合.
- E 表示整个图中所有边的集合.
- $G = (V, E)$,表示整个图.
- s 表示网络的源点, t 表示网络的汇点.
- 对于每条边 (u, v) ,有一个容量 $c(u, v)$ ($c(u, v) \geq 0$)
- 如果 $c(u, v) = 0$, 则表示 (u, v) 不存在在网络中。
- 如果原网络中不存在边 (u, v) , 则令 $c(u, v) = 0$
- 对于每条边 (u, v) ,有一个流量 $f(u, v)$.



一个简单的例子.网络可以被想象成一些输水的管道.括号内右边的数字表示管道的容量,左边的数字表示这条管道的当前流量.

网络流的三个性质

- 1、容量限制: $f[u,v] \leq c[u,v]$
- 2、反对称性: $f[u,v] = -f[v,u]$
- 3、流量平衡: 对于不是源点也不是汇点的任意结点, 流入该结点的流量和等于流出该结点的流量和。

结合反对称性, 流量平衡也可以写成:

$$\sum_{u \in V} f(v, u) = 0$$

只要满足这三个性质, 就是一个合法的网络流。

最大流问题

- 定义一个网络的流量（记为 $|f|$ ） = $\sum_{v \in V} f(s, v)$
- 最大流问题，就是求在满足网络流性质的情况下， $|f|$ 的最大值。

残量网络

- 为了方便算法的实现，一般根据原网络定义一个残量网络。其中 $r(u,v)$ 为残量网络的容量。
- $r(u,v) = c(u,v) - f(u,v)$
- 通俗地讲：就是对于某一条边（也称弧），还能再有多少流量经过。
- G_f 残量网络, E_f 表示残量网络的边集.

原网络 (a,b)表
示(流量f,容量c)

例1

图1

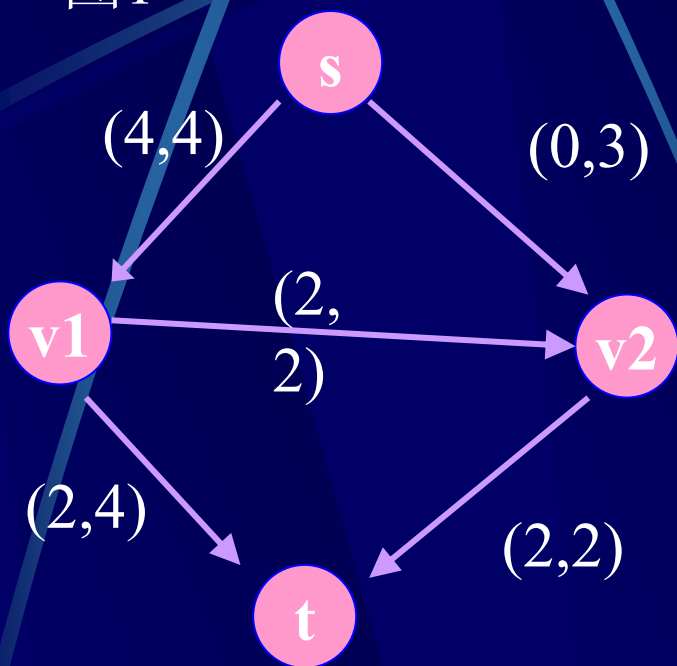
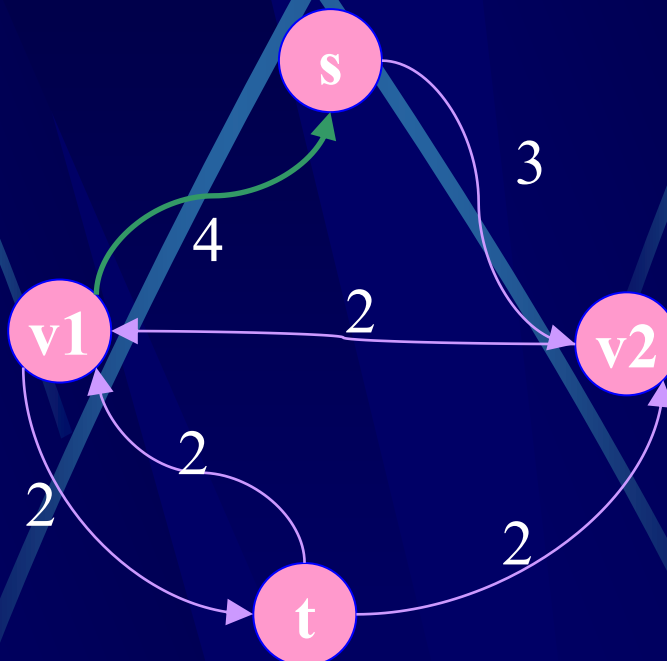
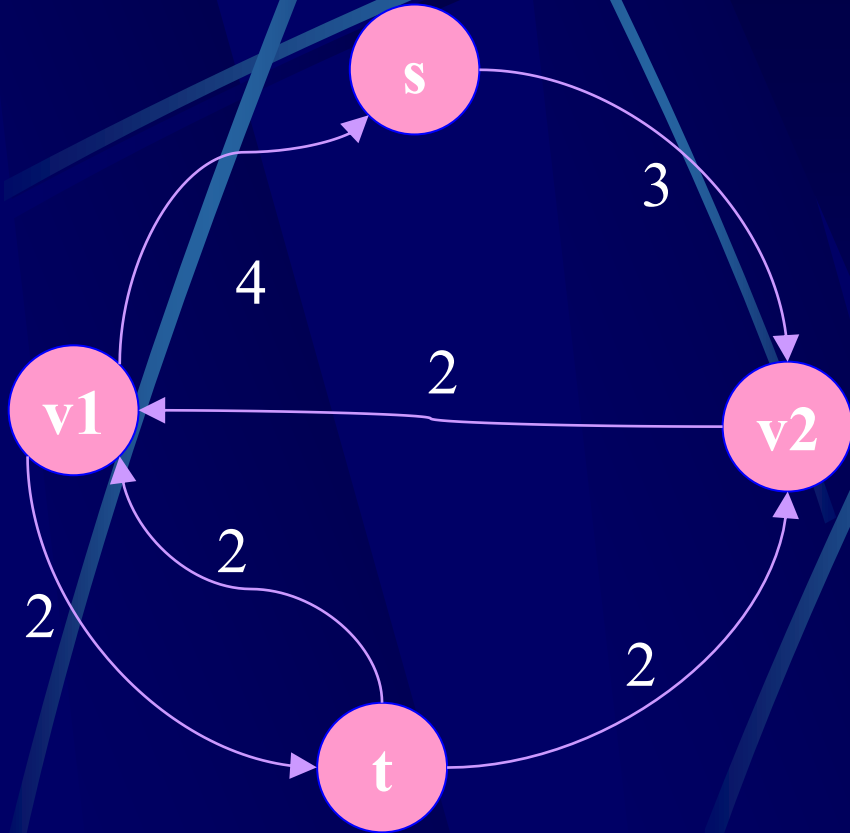


图2



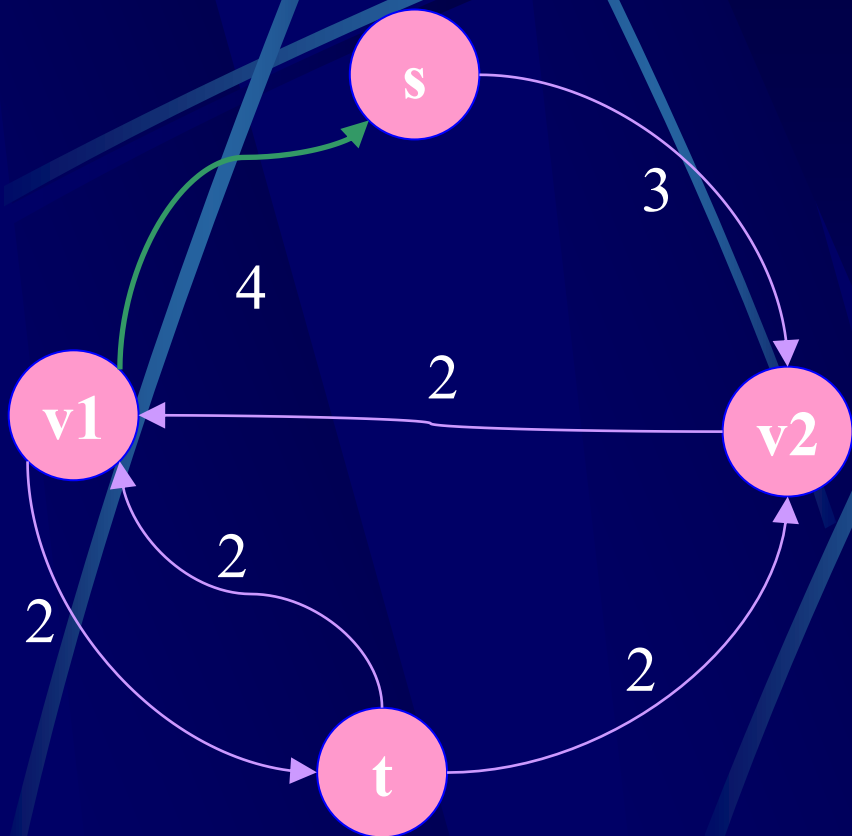
残量网络（如果网络中一条边的容量为0,则认为这条边不在残量网络中。 $r(s,v1)=0$,所以就不画出来了。另外举个例子： $r(v1,s) = c(v1,s) - f(v1,s) = 0 - (-f(s,v1)) = f(s,v1) = 4$ 。

例1



- 从残量网络中可以清楚地看到:
- 因为存在边 $(s, v2) = 3$, 我们知道从 s 到 $v2$ 还可以再增加 2 单位的流量;
- 因为存在边 $(v1, t) = 2$, 我们知道从 $v1$ 到 t 还可以再增加 2 单位的流量。

后向弧



- 其中像($v1, s$)这样的边称为后向弧,它表示从 $v1$ 到 s 还可以增加4单位的流量。
- 但是从 $v1$ 到 s 不是和原网络中的弧的方向相反吗? 显然“从 $v1$ 到 s 还可以增加4单位流量”这条信息毫无意义。那么,有必要建立这些后向弧吗?

为什么要建立后向弧

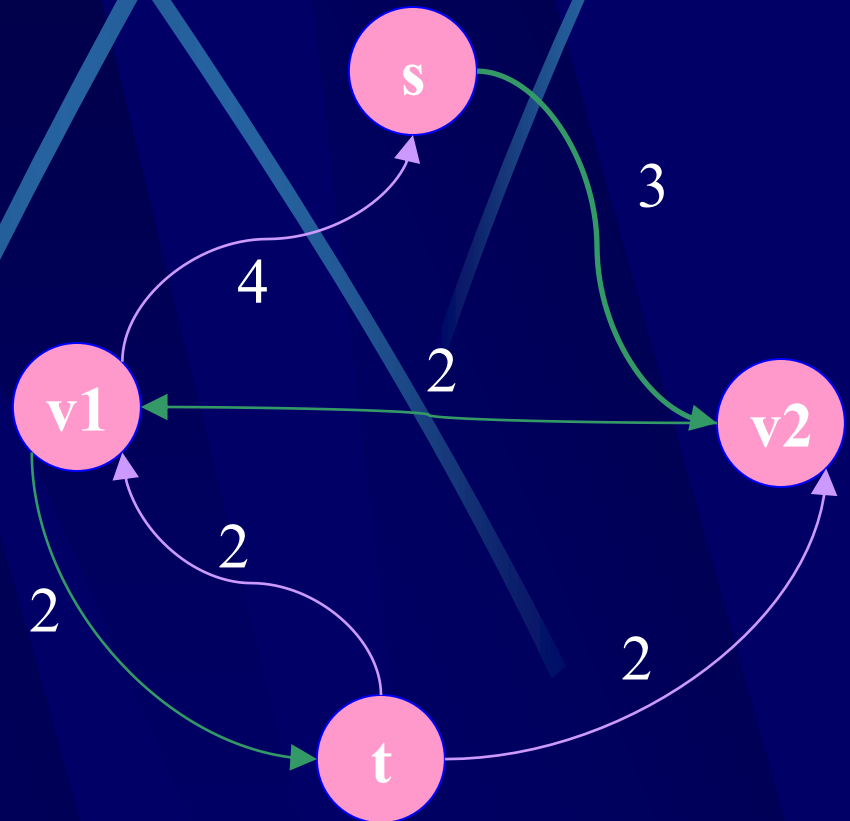
- 显然，例1中的画出来的不是一个最大流。
- 但是，如果我们把 $s \rightarrow v2 \rightarrow v1 \rightarrow t$ 这条路径经过的弧的流量都增加2,就得到了该网络的最大流。
- 注意到这条路径经过了一条后向弧: $(v2, v1)$ 。
- 如果不设立后向弧，算法就不能发现这条路径。
- 从本质上说，后向弧为算法纠正自己所犯的错误提供了可能性，它允许算法取消先前的错误的行为（让2单位的流从 $v1$ 流到 $v2$ ）

为什么要建立后向弧

- 当然,可以把上面说的情况当成特殊情况来处理。但使用后向弧可以使编程简单许多。
- 注意,后向弧只是概念上的,在程序中后向弧与前向弧并无区别。

增广路

- 增广路定义：在残量网络中的一条从 **s** 通往 **t** 的路径，其中任意一条弧 (u,v) ，都有 $r[u,v] > 0$ 。
- 绿色的即为一条增广路。



增广路算法

- 增广路算法：每次用**BFS**找一条最短的增广路径，然后沿着这条路径修改流量值（实际修改的是残量网络的边权）。当没有增广路时，算法停止，此时的流就是最大流。
- 下面证明增广路算法的正确性。

将 f, c, r 的定义域扩展为点集

- (在以后的叙述中,大写字母 X, Y, S, T 一般均表示点集)
- 点集间的流量和: $f(X, Y) = \sum_{x \in X} \sum_{y \in Y} f(x, y)$
- 即: X 中的任意一点与 Y 中的任意一点组成的所有边上的流量之和.(边的方向为从 X 中的结点到 Y 中的结点)
- c, r 等函数都有类似的定义.(点集间的容量和、点集间的残量网络容量和)

结论1

- 1. $f(X, X) = 0$ (由流量反对称性)
- 2. $f(X, Y) = -f(Y, X)$ (有流量反对称性)
- 3. $f(X \cup Y, Z) = f(X, Z) + f(Y, Z)$ (显然)
- 4. $f(X, Y \cup Z) = f(X, Y) + f(X, Z)$ (显然)

最大流最小割定理

- 网络流中这三个条件等价（在同一个时刻）：
 - 1、 f 是最大流
 - 2、残量网络中找不到增广路径
 - 3、 $|f| = c(S,T)$

- 1、**f**是最大流
- 2、残量网络中找不到增广路径
- 3、 **$|f| = c(S,T)$**

● 1 $\rightarrow$ 2证明: 显然.假设有增广路径, 由于增广路径的容量至少为1,所以用这个增广路径增广过后的流的流量肯定要比**f**的大,这与**f**是最大流矛盾.

割的定义

- 一个割(S, T)由两个点集 S, T 组成.
 - $S + T = V$
 - s 属于 S .
 - t 属于 T .
-
- 提出割的定义,是为后面的证明作铺垫.

结论2(点集总流量为零)

- 不包含s和t的点集,于它相关联的边上的流量之和为0.

- 证明: $f(X, V) = \sum_{x \in X} [\sum_{y \in Y} f(x, y)]$
 $= \sum_{x \in X} [0]$ (由流量平衡)
 $= 0$

结论3

- 任意割的流量等于整个网络的流量.
- 证明:
- $f(S,T) = f(S,V) - f(S,S)$ (由辅助定理1)
 $= f(S,V)$ (由辅助定理1)
 $= f(S,V) + f(S - s,V)$ (同上)
 $= f(s,V)$ (由辅助定理2)
 $= |f|$ (由 $|f|$ 的定义)

结论4

- 网络的流量小于等于任意一个割的容量.(注意这个与辅助定理3的区别.这里是容量)

- 即 $|f| \leq c(S,T)$

- 证明: $|f| = f(S,T) =$

$$\sum_{x \in S} \sum_{y \in T} f(x, y)$$

(由定义)

$$\leq \sum_{x \in S} \sum_{y \in T} c(x, y)$$

(由流量限制)

$$= c(S,T)$$

1、 f 是最大流

2、残量网络中找不到增广路径

3、 $|f| = c(S,T)$

- 2 $\rightarrow$ 3证明: 定义 $S = s \cup \{v \mid \text{在残量网络中 } s \text{ 到 } v \text{ 有一条路径}\}$; $T = V - S$. 则 (S,T) 是一个割.
- $|f| = f(S,T)$ (由辅助定理3)
- 而且, $r(S,T) = 0$. 假设不为0, 则在残量网络中, 两个集合间必定有边相连, 设在 S 的一端为 v , 在 T 的一端为 u . 那么, s 就可以通过 v 到达 u , 那么根据 S 的定义, u 就应该在 S 中. 矛盾.

所以, $|f| = f(S,T) = c(S,T) - r(S,T) = c(S,T)$

- 1、**f**是最大流
- 2、残量网络中找不到增广路径
- 3、 **$|f| = c(S,T)$**

● 3 $\rightarrow$ 1证明:

$|f| \leq c(S,T)$ (辅助定理4)

因为我们已经有 $|f| = c(S,T)$,如果最大流的流量是 $|f|+d$ ($d>0$),那么 $|f|+d$ 肯定不能满足上面的条件.

增广路算法的正确性

- 如果最大流最小割定理不能从2推出3,那么存在这样一种可能性:
尽管找不到增广路径了,但由于前面的错误决策,导致 f 还没有到达最大流,却不能通过修改当前流来得到最大流.
- 但由于最大流最小割定理的三个条件互相等价(1- $\rightarrow$ 2,2- $\rightarrow$ 3,3- $\rightarrow$ 1),一个流是最大流当且仅当它没有增广路径.

增广路算法的效率

- 设 $n = |V|$, $m = |E|$
 - 每次增广都是一次BFS,效率为 $O(m)$
 - 所以,总共的时间复杂度为 $O(m*f_*)$
 - 其中 f_* 为增广次数.
-
- 怎么求 f_* ?

f_*

- 对于随机数据, f_* 的值与 n 比较接近. 当 m 不太大也不太小时, f_* 的值较大.
- (我出随机数据的方法是: 固定地为源点和汇点连上一些边, 然后随机生成中间的边. 中间的边保证边的两个端点的编号相差不太大. 这与不少题目转成网络流后形成的图相似)

f^* 的理论 upper 界

- 考虑每一次增广,至少有一条边的 $r(u,v)$ 值等于增广路径的流量.称这些边为临界边.增广之后,这条临界边就在残量网络中消失.
- 假设一条临界边对应一次增广(事实上很难达到这样),令每条边成为临界边的次数为 $k(u,v)$,则有 $f^* = O(m \cdot k)$.
- k 的上界?

k的上界

- 如果要想让一条曾经的临界边 (u,v) 再次成为临界边,则必须有一条增广路径包含边 (v,u) .因为每次增广之后临界边就消失,要让他再次成为临界边至少要让他再次在残量网络中出现,即 (v,u) 要被增广.
- 结合上面的结论可以证明,当算法取的增广路总是残量网络中的最短路,任意一条边成为临界边的次数至多为 $n/2-1$.
- 因此,增广路算法的效率为 $O(f \cdot m) = O(km^2) = O(nm^2)$. (这只是个上界,一般情况是达不到的)
- 备注中为增广路算法我的代码实现。数组u是残量网络的容量。



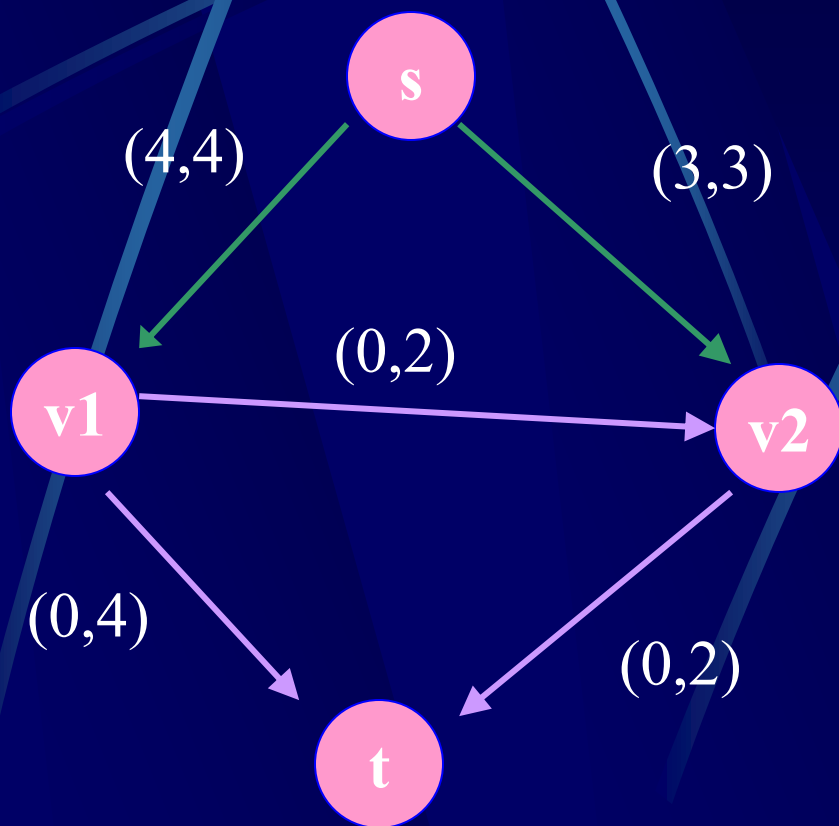
预流推进算法

下面将介绍一个更直观且时间效率更优的算法.

一个直观的想法

- 如果给你一个网络流,让你手算出它的最大流,你会怎么算?
- 一般人都尝试着从源点出发,让每条边的流量尽可能得大,然后一点点往汇点推,直到遇到一条比较窄的弧,原先的流量过不去了,这才减少原先的流量.

例2.一个直观的想法

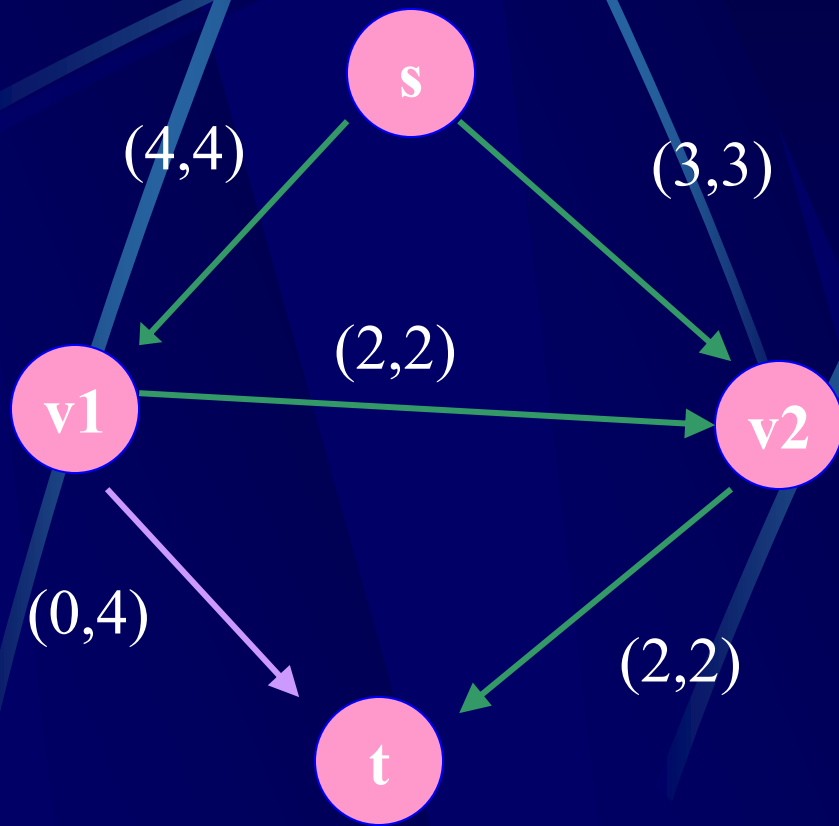


大致的思路：从源点出发，逐步推进。

称当前状态下不满足流量平衡的结点为“溢出的结点”。(对于结点 $u, f(V, u) > 0$)

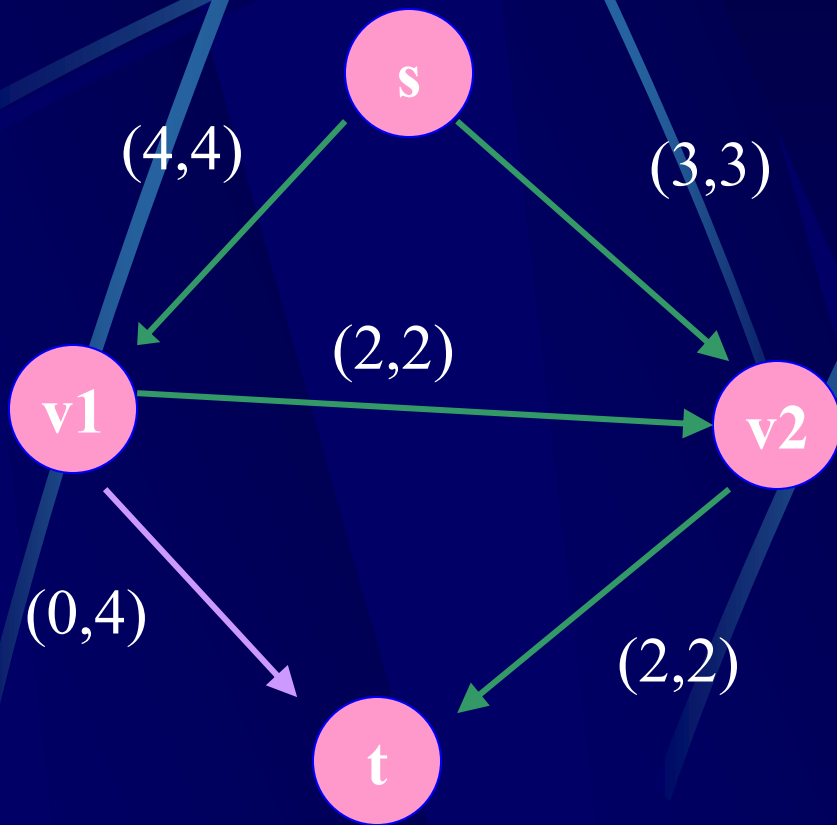
令 $e(u) = f(V, u)$,称为 u 点的赢余，直观地描述，就是“流入的比流出的多多少”。 $e(v1)=4, e(v2)=3$ 。不断将溢出的结点中的赢余往后继点推进,直到赢余都聚集在 t 。

例2. 一个直观的想法



如果多推了一些流量, 我们可以再把它推回来. (如 $e(v2)=3$, 但这3个单位的赢余已经没地方去了, 只能推回来.)(沿着后向弧)这副图是原网络而不是残量网络, 因此没把后项弧画出来)

程序没有全局观?!



此时 $e(v2)=3$.正确的回推法是往 $(v2,s)$ 推1,往 $(v2,v1)$ 推2,然后使得这2个单位的赢余可以从 $(v1,t)$ 推到 t 上。
但程序没有全局观,它万一往 $(v2,s)$ 推了3个单位怎么办?我们总不能尝试所有的可能性吧,那样就变成搜索了.

引导机制

- 把流推错可能导致产生的流不是最大流.
- 我们需要有一个能引导流的推进方向的机制,当它发现我们先前的推进是错误的時候,能沿着正确的后向弧回推回来.
- 由于建立了后向弧,正推与回推在程序中并无却别, 都是在推残量网络中的一条边.

高度标号的引导作用

- 高度标号就是这样的一个引导机制.
- 我们规定,如果一个结点溢出了,那么他的多余的流量只能流向高度标号比自己低的结点.(“水往低处流”)
- 当然,高度标号不可能事先知道往哪些方向推才是正确的.它将按情况动态改变自己的值,从而正确地引导流向.

重标号操作

- 当一个结点有赢余(溢出了), 周围却没有高度比它低的结点时候,我们就用重标号操作使它的标号上升到比周围最低的结点略高一点,使他的赢余能流出去.
- 赢余千万不能困在某个结点里.对于任意一个非源非汇的结点, 有赢余就意味着它不满足流量平衡, 也就意味着整个网络流不是一个真正合法的网络流。

重标号操作

- 对于例2的这种情况, v_2 中过多的赢余最终会沿着 (v_2, v_1) 、 (v_2, s) 流回去(虽然他们一开始流错了方向,但后来又被回推,等于说是被改正了)。
- 只有当非源非汇的结点中的赢余全部流到汇点或流回源点后, 这个流才重新合法。

高度函数

- 高度函数 $h(v)$ 返回一个 v 的高度标号。
- 高度函数有三个基本条件:
- $h(s) = |V|$
- $h(t) = 0$
- 对于 E_f (残量网络)中的每一条边 $(u,v), (r(u,v)>0)$
$$h(u) \leq h(v) + 1$$
- 这第三个条件看上去有些奇怪.既然 $r(u,v)>0$,那就表示从 u 到 v 还可以增加流量,那 $h(u)$ 就应该比 $h(v)$ 高才对.的确,我们后面还将规定,只有在 $h(u)>h(v)$ 的时候才能应用推进操作(将一个结点的盈余推进到另一个结点的操作).而高度函数为了满足其合法性,还要满足上述的这三个条件.后面我们将利用这三个条件证明预流推进算法的正确性。

高度函数的条件的实质

- $h(u) \leq h(v) + 1$.
- 这个条件实质上是要求高度不能下降的太快，即水只能在高度相差不多的地方缓缓流过，不能像瀑布一样从很高的地方流到很低的地方。(否则就有流错的危险)
- 这和A*算法中的启发函数必须“相容”的条件类似。h函数的缓慢下降，保证了算法的正确性。后面我们将看到这个条件的作用。

●两个关键操作

- 推进操作(将一个结点的盈余推到另一个结点)
- 重标号操作(更改一个结点的高度值,使其的盈余能朝着更多的地方流动)

推进操作

- 使用对象：一条边 (u,v)
- 使用条件：
 - $e(u) > 0$, $r(u,v) > 0$, $h(u) = h(v) + 1$
 - (u 溢出, (u,v) 在残量网络中, 两者的高度差为1)
- 推进量为 $e(u)$ 与 $r(u,v)$ 的最小值。
- 推进时同时更改相关的 r 与 e 的值。

推进操作 伪代码

- Procedure Push(u,v)
 - $X \leftarrow \min\{ e(u), r(u,v) \}$
 - Dec(r(u,v), x) Inc(r(v,u), x)
 - Dec(e(u), x) Inc(e(v), x)

重标号操作

- 使用对象： 一个结点 u
- 使用条件：
- 结点 u 溢出； 残量网络中周围所有的点的高度都不比它低。
- **Relabel(u)**
 - $u(u) = \min\{ h(v) \mid (u,v) \text{ 是残量网络总的边} \} + 1$
- 使用了重标号操作后,至少存在一个 (u,v) 满足 $h(u)=h(v)+1$.

预流初始化(Init-Preflow)

- 一开始的时候，我们要让和源点 s 相关连的边都尽可能的充满。但由于 s 没有溢出，不符合推进操作的使用条件，我们需要另写一段初始化的代码。还得做的一件事是初始化高度函数。
- $h(s) = n$ $h(v) = 0$ ($v \neq s$)
- 对于所有与 s 相关联的点 v ，
 - $\text{Inc}(e(v), c(s,v)), \text{Dec}(e(s), c(s,v))$
 - 将边 (s,v) 反向，变成 (v,s) （在残量网络中）。
- 初始化过后， $e(s)$ 变成负数。

结论5

- 对于一个溢出的结点，两个关键操作（推进和重标号）能且只能应用一个。
- 证明：对于一个溢出的结点 u ，和所有与他相关联的点 v (u,v 在残量网络中存在)，必然有 $h(u) \leq h(v) + 1$. (由高度函数的定义).
- 根据 v 分成两种情况: 1). 所有 v 都有 $h(u) < h(v) + 1$ 2). 至少存在一个 v , 使得 $h(u) = h(v) + 1$. 而1)2)互为否命题, 不能同时成立或同时不成立. 那么1)对应重标号, 2)对应推进, 两者必能应用一个且只能应用一个.

一般的预流推进算法

- 由辅助定理5,得到了一个一般的预流推进算法.(好短)
- **Init-Preflow**
- **While** 存在一个溢出的结点
 - 选一个结点,应用相应的关键操作(推进或重标号).
- 当不存在溢出结点时(s, t 不算),算法结束,得到一个可行流,并且还是最大流.

预流推进算法的正确性

- 预流只是不满足流量平衡,网络流的前两条性质---容量限制和反对称性它还是满足的.当不存在溢出结点时,流量平衡也满足了.
- 所以,当算法结束时,我们得到一个可行流(合法流).
- 为什么他是一个最大流呢?
- 下面先看几个结论:

结论6(结点高度永不下降)

- 只有重标号操作能更改结点的高度标号.
- 在重标号操作应用前,必有 $h(u) \leq h(v)$ (u, v 相邻). 令 $v_0 = h$ 值最小的一个 v ,则 $h(u) = h(v_0) + 1 \geq h(u) + 1$.
- 所以,在重标号操作后,高度标号至少+1.

结论7

- 在算法执行过程中, h 始终是一个合法的高度函数.(满足那三个条件)
- 1). 考察一个被重标号的结点 u .
 - 设 (u, v) 存在于 E_f , v_0 是所有 v 中 h 最小的一个.
 $H'(u) = h(v_0) + 1$, 满足 $h(u)' \leq h(v_0) + 1$, 而 $h(v_0) \leq h(v)$, 所以 $h'(u) \leq h(v) + 1$.
 - 设 (w, u) 存在于 E_f , 则
 $h(w) \leq h(u) + 1 \leq h'(u) + 1$. 仍旧满足.

结论7

- 在算法执行过程中, h 始终是一个合法的高度函数.(满足那三个条件)
- 2). 考察一个被推进的边 (u, v) .
 - (v, u) 可能是在这次推进之后才出现在 E_f 中. 它的出现使得新增了一个限制条件: $h(v) \leq h(u) + 1$. 不过, 这显然是满足的, 因为推进操作的使用条件是 $h(u) = h(v) + 1$. 那么 $h(v) = h(u) - 1 \leq h(u) + 1$

结论8(预流中无增广路)

- 当 h 是一个合法的高度函数时, G_f 中始终不存在增广路.(这个定理展示了 h 的条件的重要性和巧妙性)
- 证明:假设存在增广路 $p=(v_0, v_1, \dots, v_k)$,其中 $v_0=s, v_k=t$.因为增广路径中无重复点, $k+1 \leq |V|$,即 $k < |V|$.

结论8(预流中无增广路)

$$(v_0, v_1) \in E_f$$

$$(v_1, v_2) \in E_f$$

.....

.....

$$(v_{k-1}, v_k) \in E_f$$

→

$$h(v_0) \leq h(v_1) + 1$$

$$h(v_1) \leq h(v_2) + 1$$

.....

.....

$$h(v_{k-1}) \leq h(v_k) + 1$$

相加得:

$$h(s) \leq h(t) + k$$

$$\leq 0 + k \leq k$$

→ 而 $k < |V|$, 所以
 $h(s) < |V|$.

而根据定义, $h(s) = |V|$.
矛盾.

预流推进算法的正确性

- 当有溢出结点时,根据结论5,必定可以在它上面施加一个操作.
- 当算法停止时,因为无溢出结点,所以当前流是一个合法流,而根据结论8, G_f 中始终不存在增广路.根据最大流最小割定理,当 G_f 中不存在增广路时,当前流是最大流.
- (算法执行了一半时虽然也没有增广路,但由于它不是一个合法流,前面的诸多定理都不成立).
- 算法的最优性的保证者:
对于所有在 E_f 中的 (v,u) , 均有 $h(v) \leq h(u) + 1$

更好的预流推进算法

- 前面的一般预流推进算法可以实现为 $O(n^4)$. 其瓶颈是非饱和推进.(非饱和推进是指在推进之后仍旧没有使 (u,v) 消失的推进.)
- 通过恰当地安排关键操作的顺序, 可以使总的推进(主要是非饱和推进)和重标号的次数减少. 接下来的 **relabel-to-front** 算法就用了这个思想.

Relabel-to-front

- relabel-to-front算法维护一个结点列表, 然后依次检查列表中的结点. 检查的过程就是:一口气将所有的赢余推给周围的人. 如果在检查的时候这个结点被relabel了, 那么他就被移到整个列表的最首部, 并且重新从列表首部开始检查结点.
- 通过这样恰当地安排操作顺序(一次性把某个结点所有的赢余全部推掉), 复杂度降到了 $O(n^3)$.

一些定义

- 如果满足下面的两个条件,称 (u,v) 为可行弧:
 - $r(u,v) > 0$
 - $h(u) = h(v) + 1$
- 可行边集 $E_{f,h}$:所有由可行弧组成的集合。
- 可行网络 $G_{f,h} = (V, E_{f,h})$

结论9,10

- 结论9:可行网络中无环.(和结论8的证明类似,弄一堆式子然后叠加一下,导出矛盾)
- 结论10:推进操作永远不会新增可行弧,却可能使原有的可行弧消失.(根据可行弧的定义显然)

结论11

- 在 u 被重标号之后:

- 1).至少有一条可行弧离开 u .

显然.设 v_0 是 u 的邻居中 h 值最小的那一个,则 (u, v_0) 必定是一条可行弧.

- 2).不可能有可行弧进入 u .

假设有一条 (w, u) .则 $h(w) = h'(u) + 1$.根据辅助定理6, relabel操作至少将结点的 $h+1$,所以 $h(w) > h(u) + 1$.根据高度函数必须满足的条件, (w, u) 在relabel前不在 E_f 中.而relabel操作只改变可行网络不改变残量网络, (w, u) 不可能在relabel前存在于 E_f 而之后就不存在.

当前弧

- 每个结点有一个邻居列表和有一个“当前弧”的指针,保存当前检查到邻居列表中的哪一条弧了。初始化时,“当前弧”指向与该结点相连的第一条边.邻居列表保存的是所有可能成为可行弧的弧.
- 当再次调用检查操作时,可以从上一次检查了一半的地方继续检查.
- 具体请看下面检查操作的伪代码:

检查操作

- **Check(u)**

- **While $e(u) > 0$ do**

- **If $current(u) > degree(u)$ then** //当没有可行弧可以推进,该结点却仍旧有赢余时,重标号.

- **Relabel(u)**

- **Current(u) = 1**

- **Else**

- **If $(u, current(u))$ 是一条可行弧 then**
Push(u, current(u)) //push了之后就不能增加 **current(u)** 的值.因为这如果是一次非饱和推进,那再下一次检查时还是可以沿着这条弧做推进.

- **Else Inc(current(u))**

当前弧的正确性

- **Current**是全局变量,当某次**Check**操作结束时他的值并没有被清空.
- 比如结点u有10个邻居,上次检查到第7个,那再一次**Check(u)**的时候就只要从第7个开始检查就可以了。
- 为什么再一次检查的时候不要检查第1-6条边了?能否证明在再一次检查的时候他们一定不是可行弧?

当前弧的正确性

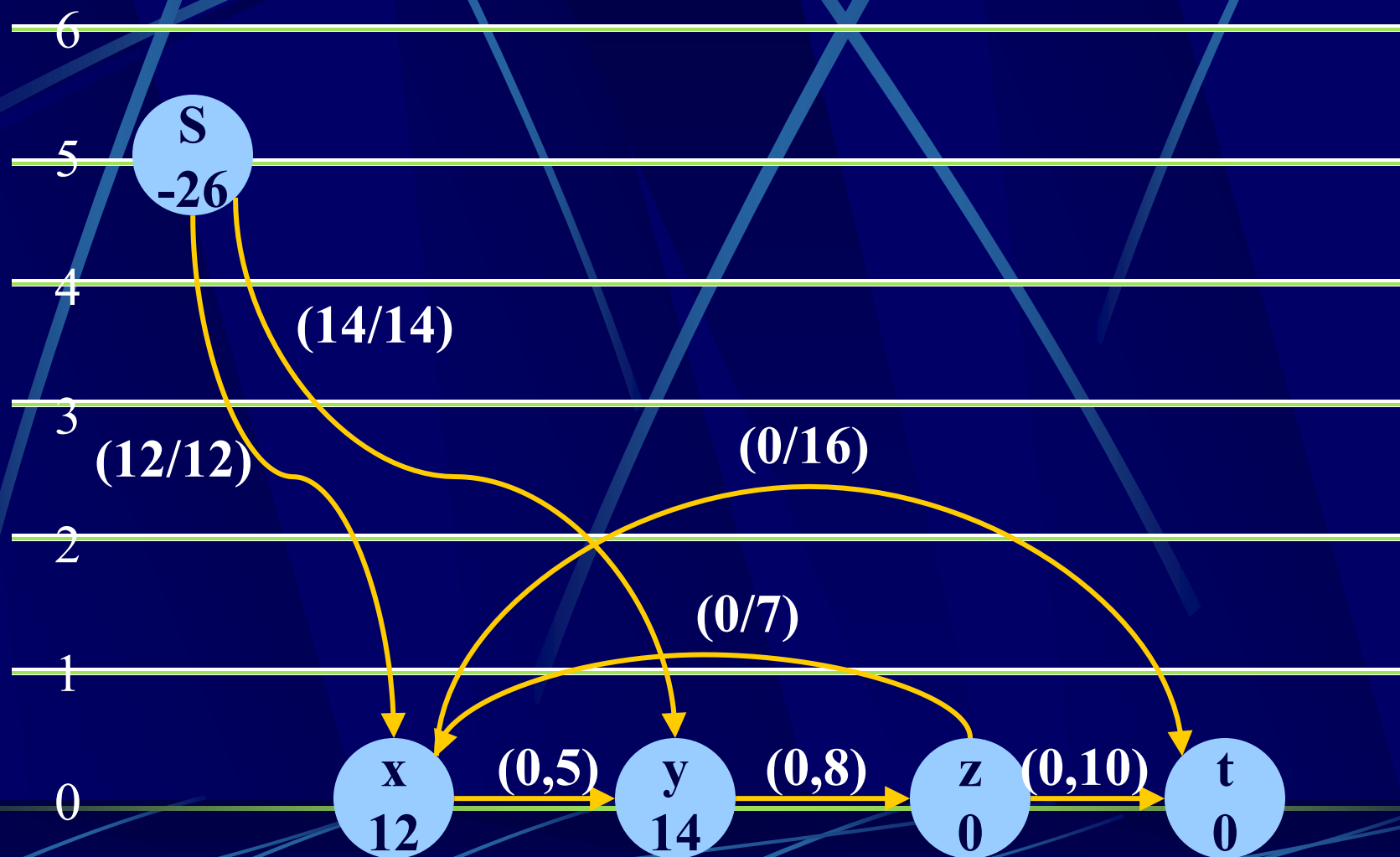
- 在relabel-to-front算法中, relabel只被Check调用.
- 当“当前弧”移动时, 移动前它指向的那条弧一定是不可行的. 而推进操作不能创造可行弧. 只有relabel可以. 两次Check之间没有relabel操作. 所以原先的不可行的弧在第二次Check之前一直是不可行的.

Relabel-to-front

- Init-Preflow
- 初始化结点(除s,t)列表L(任何顺序均可)
- 令所有u, $\text{Current}(u) = 1$
- $u \leftarrow \text{Head}[L]$
- While $u \neq \text{nil}$ do
 - $\text{Old-height} \leftarrow h(u)$
 - Check(u)
 - If $h(u) > \text{old-height}$ then 将u移到L首部
 - //如果 $h(u)$ 比原先的 h 高了,说明被relabel,移到队首.
 - $u \leftarrow \text{next}(u)$

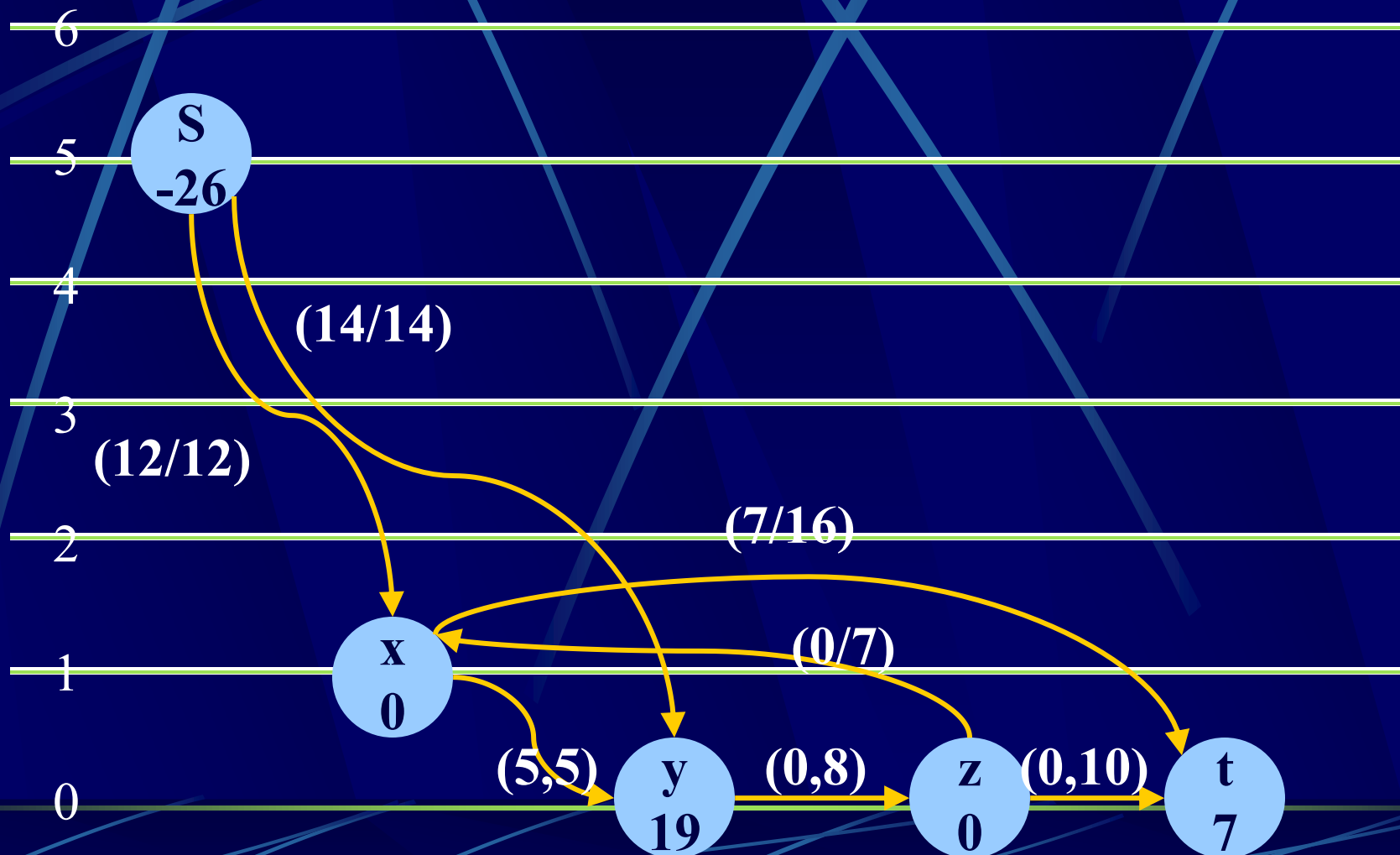
L **x** y z
 N **s** **s** **x**
 y x y
 z z t
 t

图例 (初始状态. 结点下方数字为赢余, **N** 显示的是邻居列表, **N** 中红色的是当前弧指针所在的位置.)



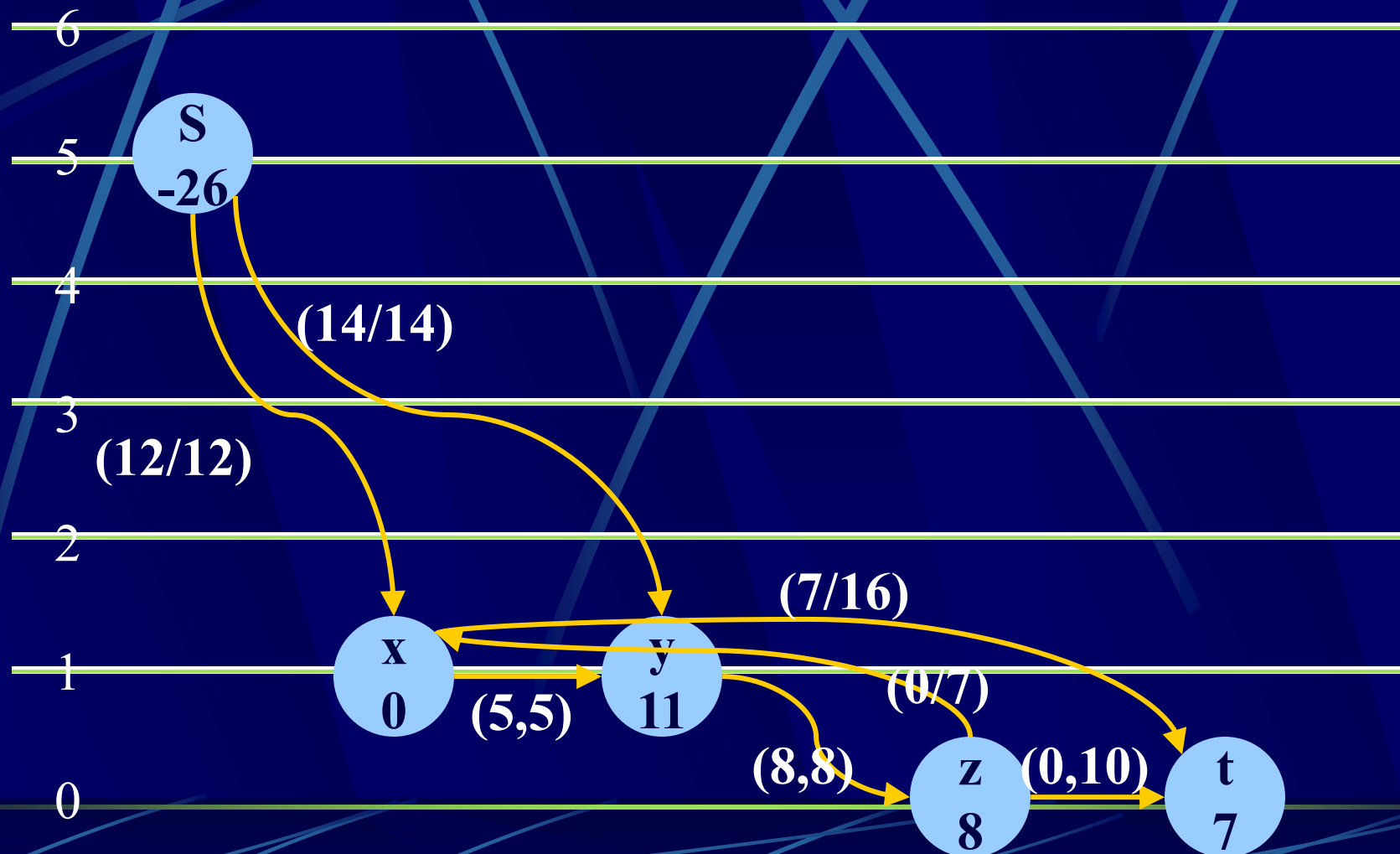
L x y z
 N s s x
 y x y
 z z t
 t

图例:**x**被检查并重标号,并被提到**L**的首部(等于没提).注意当前弧的指针移到了**t**. **x**的所有赢余推给了**y**和**t**.



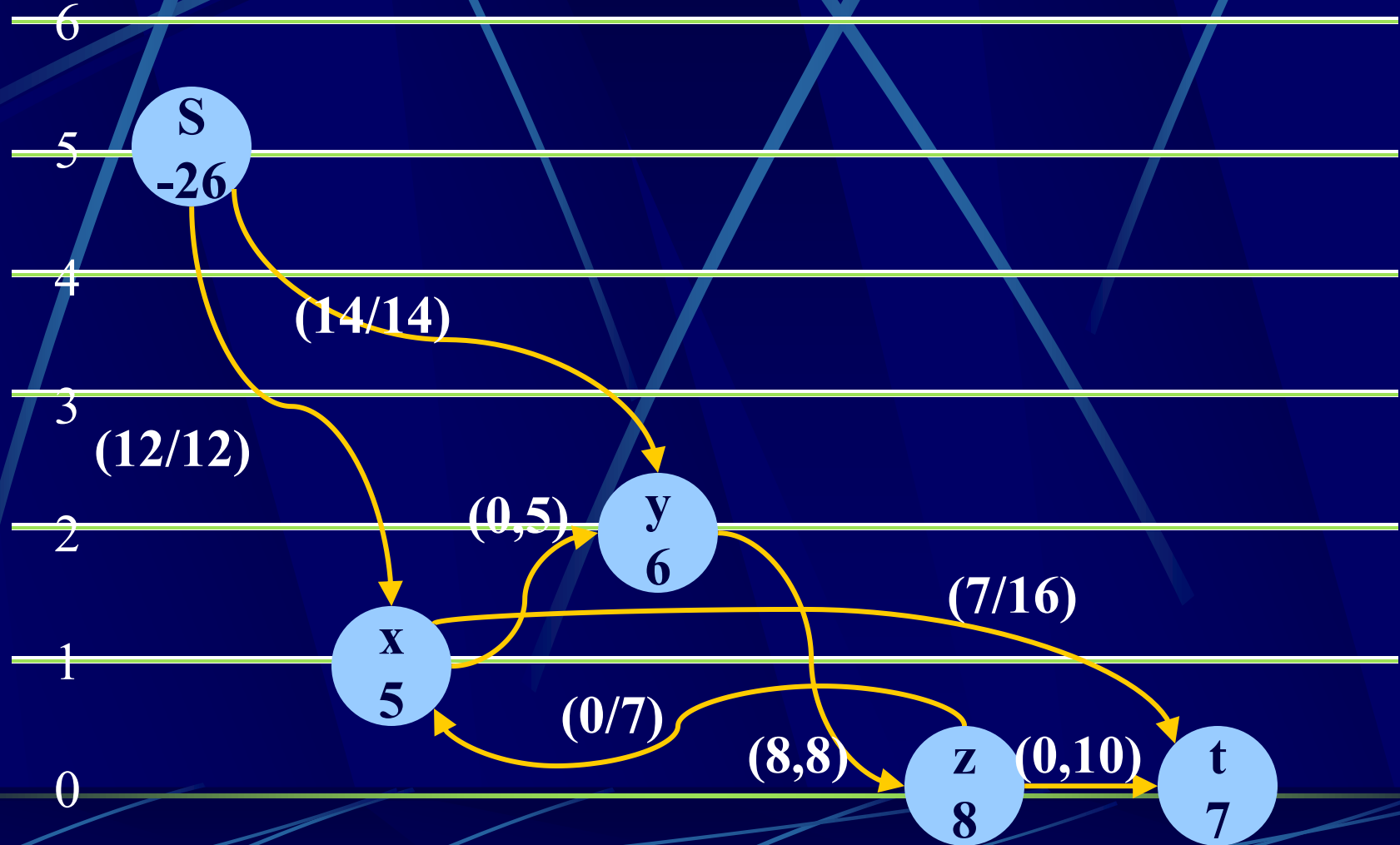
L x y z
N s s x
y x y
z z t
t

图例 :y正在被检查.将8单位的赢余推给z之后还是有剩余.



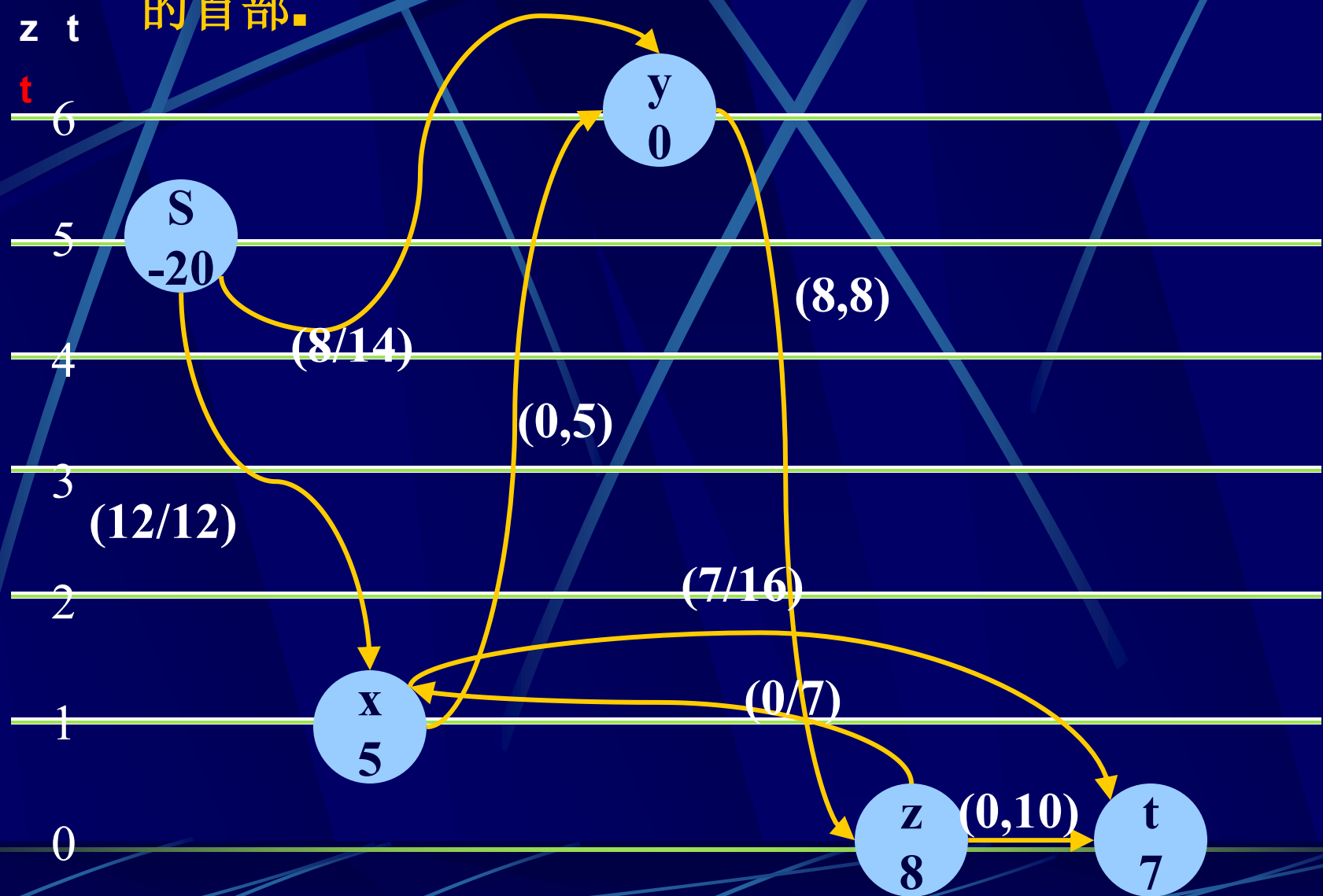
L x y z
 N s s x
 y x y
 z z t
 t

图例：一次必须把赢余全部推光。所以y被重标号,当前弧指针从头开始查找,找到(y,x)这条可行弧之后进行推进。实际上是把多推的赢余还给了x。因为 $h(u) \leq h(v) + 1$ 的保证,它没有把赢余错推给s。



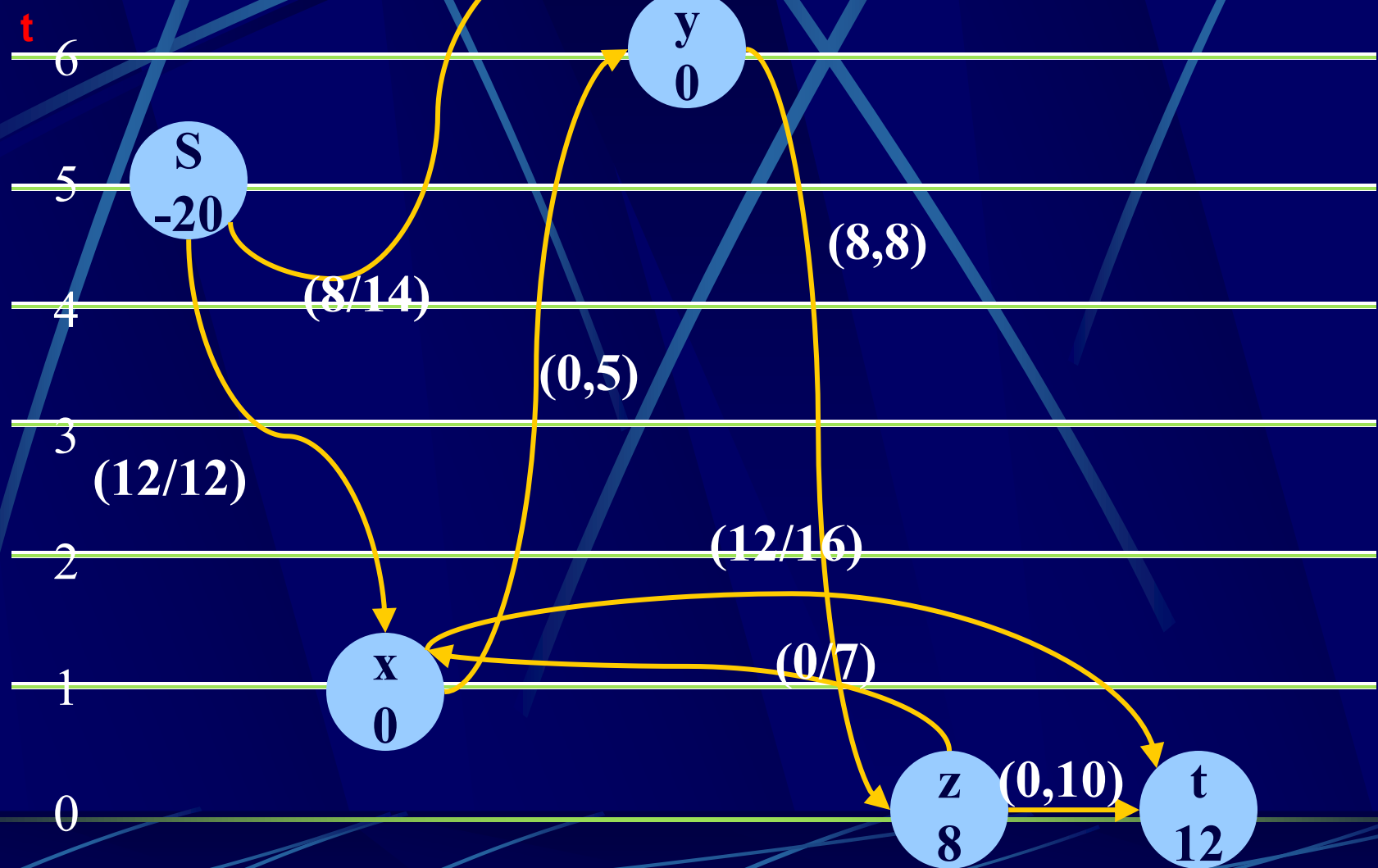
L y x z
 N s s x
 x y y
 z z t
 t

图例:**y**还是有赢余.当当前弧移动到另局列表的尾部时,**y**再一次被重标号,并把赢余还给**s**.检查结束,**y**被提到**L**列表的首部.



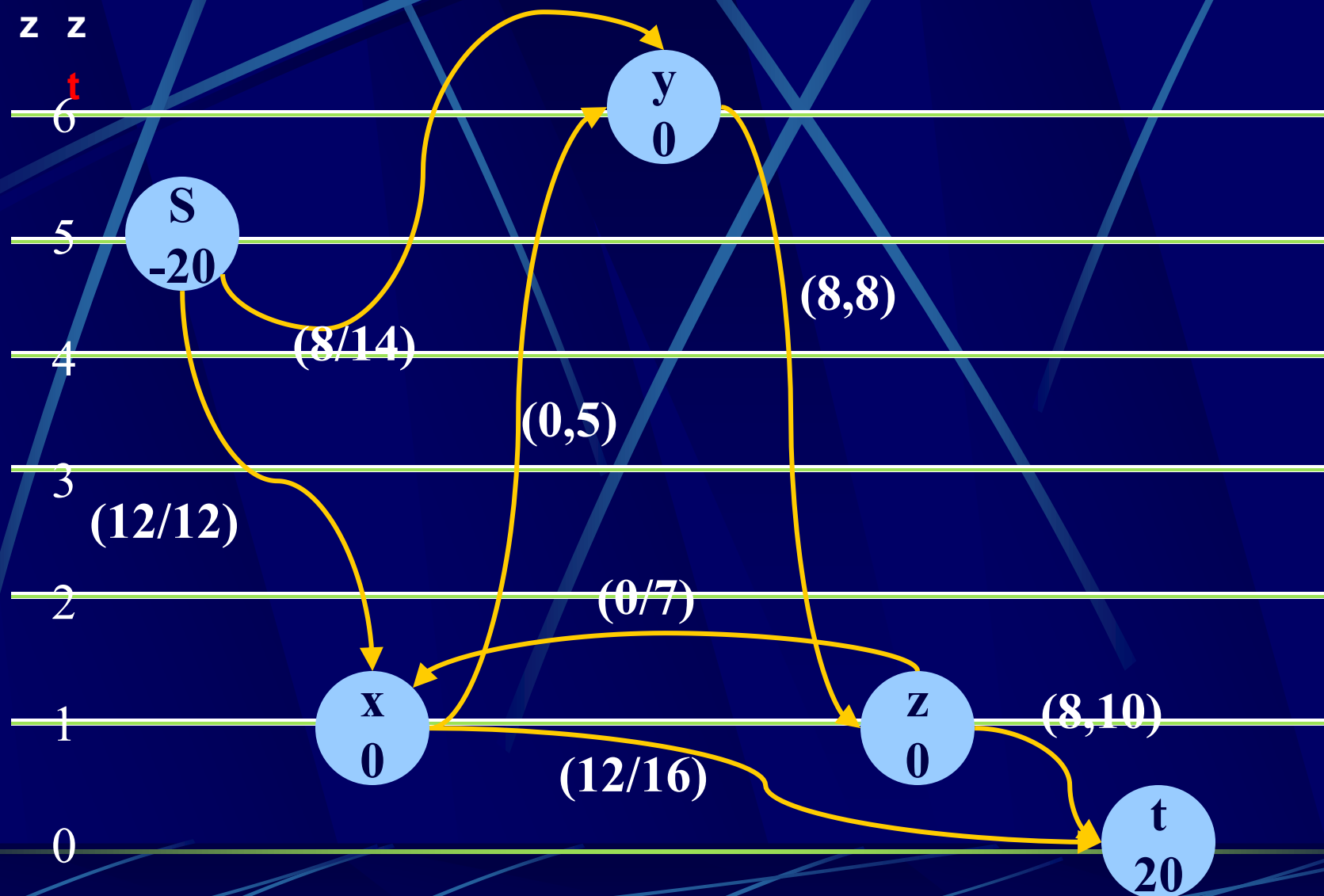
L y x z
N s s x
x y y
z z t

图例:检查x.注意x的当前弧指针已经指在t上了.
x把赢余推给t. u指针直接后移.(因为x没有被重
标号)



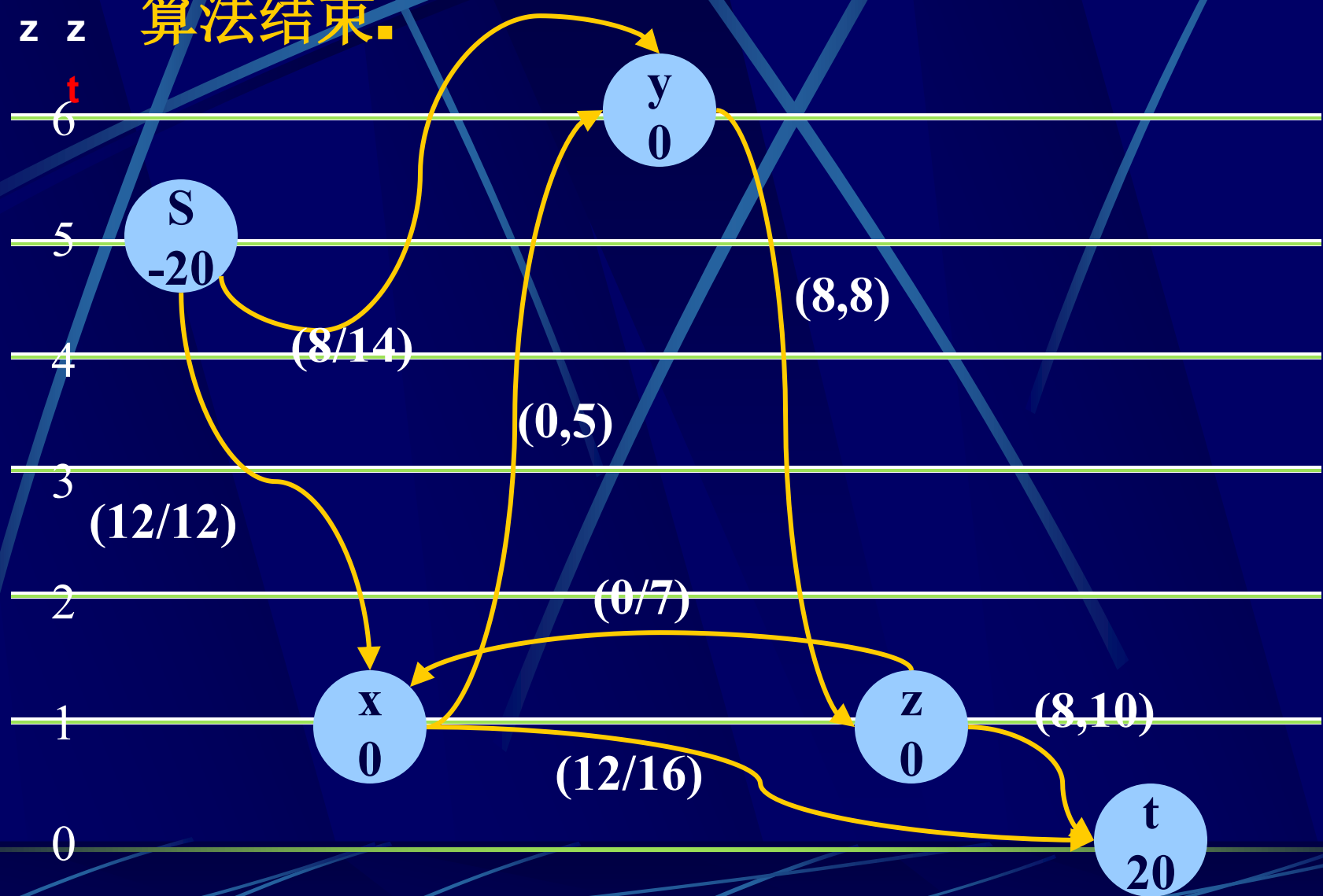
L z y x
N x s s
y x y
t z z

图例: **z**被检查并被提到列表首部.



L z y x
N x s s
y x y
t z z

图例: **u** 指针从 **y** 开始向后移动, 直到队尾也没有发现可以检查的结点(只有溢出的结点才能被检查).
算法结束.



relabel-to-front的正确性

- 前面我们已经证明了一般预流推进算法的正确性了.因此,现在只要证明,在 **relabel-to-front** 算法结束时,一般预流推进算法的结束条件也正好被满足----即没有溢出的结点.

结论11:L始终拓扑有序

- 对于G上的可行网络 $G_{f,h}$,列表L中的结点始终保持拓扑有序性.
- 一开始的时候,列表中所有结点(s,t不在列表中)的高度均为0,不存在高度差,所以不存在可行弧.这时列表显然拓扑有序.
- 一个结点被relabel之后,就被提到列表的首部.根据辅助定理11,relabel之后没有可行弧进入结点,但有可行弧离开结点,所以将结点提到列表首部仍旧使列表满足拓扑有序.
- 推进操作不能创造可行弧,因此与列表的拓扑有序性无关.

结论12

- **L中指针u之前的结点全部是非溢出结点.**
- 当一个结点被检查之后,它必定没有赢余,因此将u指针后移不影响上面的性质.
- 它自己没有赢余了,但它却可能将赢余推给了别人.如果推给在L中位置在它后面的结点不要紧.但如果它把赢余推给了在自己之前的结点呢?
- 因为L拓扑有序,他若把赢余推给了排在自己前面的结点,则必定发生了**relabel**操作.而如果有**relabel**,则它已经被提到列表的首部了.性质依然满足.
- 这就是算法名:**relabel-to-front**的由来.

Relabel-to-front

- 根据一般的预流推进算法,当没有溢出结点时算法就结束并得到一个最大流.
- 而**relabel-to-front**算法的结束条件是u指针指向L队列尾部.
- 根据辅助定理12,u以前的结点均非溢出结点.所以当u指向尾部时,所有的结点均没有溢出.
- 另外可以证明,算法的复杂度为 $O(n^3)$.

Highest-relabel

- 还可以改进.
- 经验表明,总是检查高度标号最大的结点,会有比较好的效率.
- 于是对Relabel-to-front进行了一点小修改,得到了highest-relabel算法.

分块的L列表

- 可以证明,任意结点的最大的距离标号为 $2n-1$.
- 将L列表分成 $2n$ 个块,第1块保存所有高度为0的点,第 $i+1$ 块保存高度为 i 的所有结点.
- 从最后一块开始往前找,发现一个不为空的块就把这个块里的结点全部检查掉.如果有元素被重标号了,那就将他移动到新的块里,并从那个新的块的前面开始继续往下查找.

分块的L列表

- 对于可行网络,分块L列表是拓扑有序的.
- 因为可行弧 (u,v) 要求 $h(u) = h(v) + 1$,即只有从标号高的结点指向标号低的结点.既只有从后面的块里的结点指向前面的块里的结点.所以,这种分块方法仍然保持了整个列表的拓扑有序性.因此,算法结束时没有溢出的结点.因此该算法是正确的.

分块L列表的实现

- 如果用链表，可以只占用 $O(n)$ 的空间。在内存不紧张的情况下，也完全可以用无序数组，时间效率不比链表差，虽然空间是 $O(n^2)$ 的。
- 无序数组在删除的时候，可以用：
- **Delete(i)**
 - $a[i] \leftarrow a[n]$
 - Dec(n)

更多的改进

- 回顾一下上面两个算法的正确性:
- 1). h 是一个合法的高度函数 $\rightarrow$
 G_f 不存在增广路
- 2). 同一结点 h 值保证递增 $\rightarrow$
重标号后没有可行弧进入结点 $\rightarrow$
列表 L 永远拓扑有序



$\rightarrow$ **Relabel-to-front(highest-relabel)** 算法正确.
也就是说, 只要满足 1). 2). 两条, h 的值具体怎么变化并不重要.

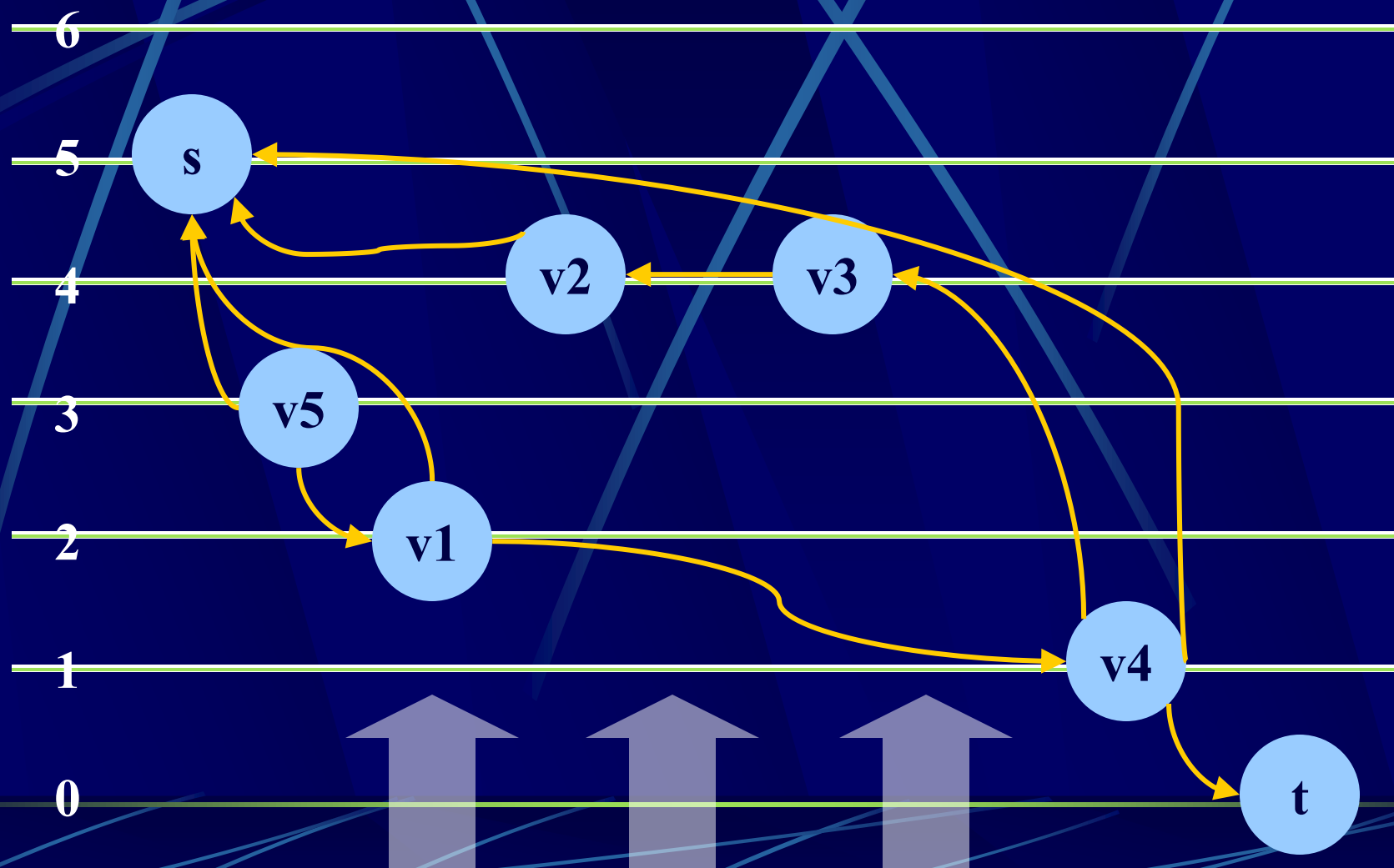
更多的改进

- 可以发现,每当重标号操作被执行,该结点的当前弧的位置就被重置,同时这个结点被往前提,然后这个结点后面的结点又全部得被检查一遍.也就是说,每次重标号操作都很新产生很多工作,我们应该尽量减少重标号的次数.
- 而 h 值的具体变化规则不影响正确性.于是,我们希望 h 值能增长得快一点.(在满足 h 函数的合法性和单调递增性的情况下)

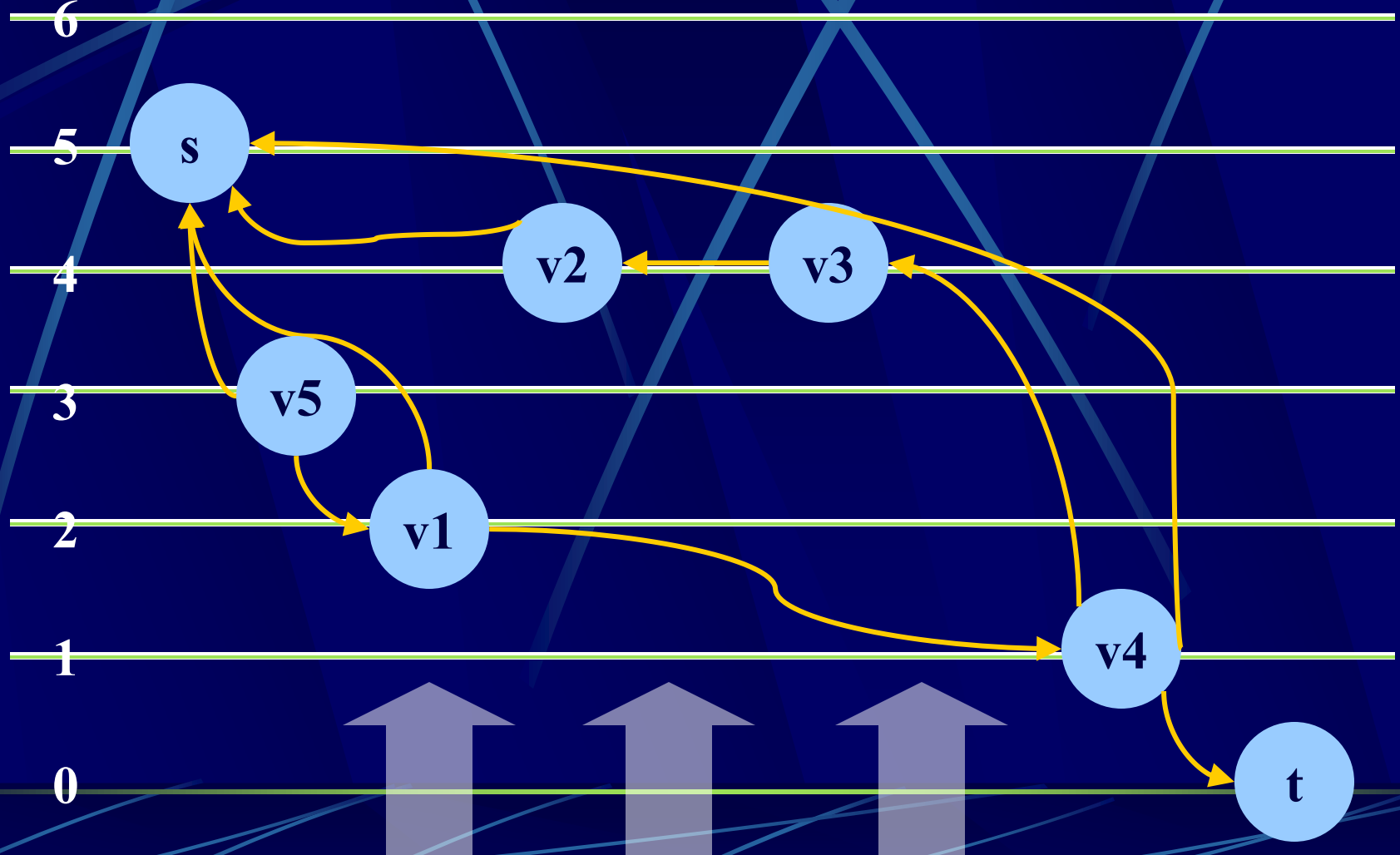
BFS预处理

- 从前面的图例中可以发现,有些结点根本没做什么事,第一件事就是重标号.我们希望能将事先将他们的初始标号算好,从而减小重标号次数.
- 从汇点 t 开始做BFS,将经过的结点(s 除外)的高度标号预设为该结点到 t 的最短路径.
- 可以证明,用这个预处理优化过的算法仍然是正确的.

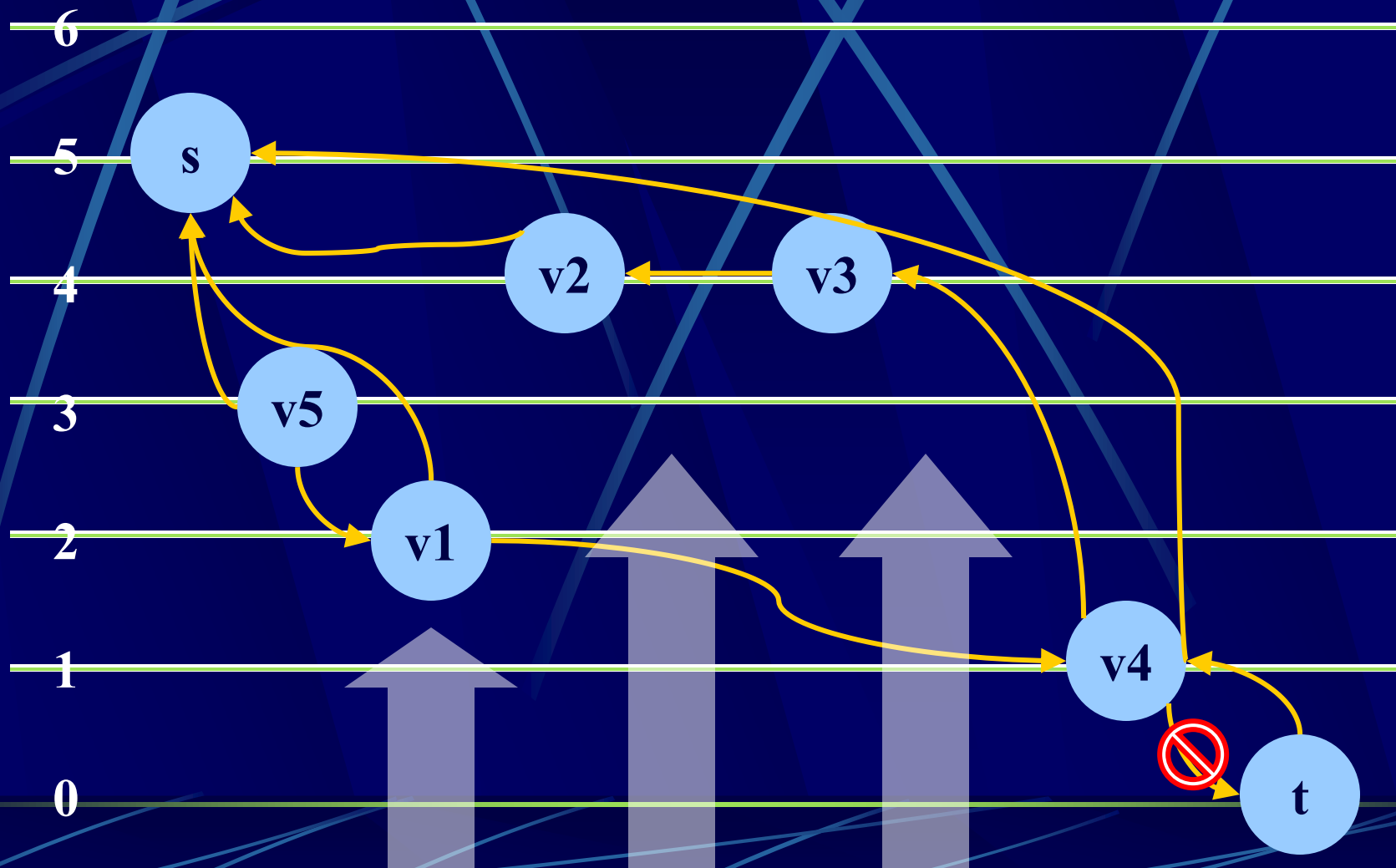
如图所示的残量网络.因为我们希望 h 增长得尽量快,因此想象在图的底部有风在从低往高吹.



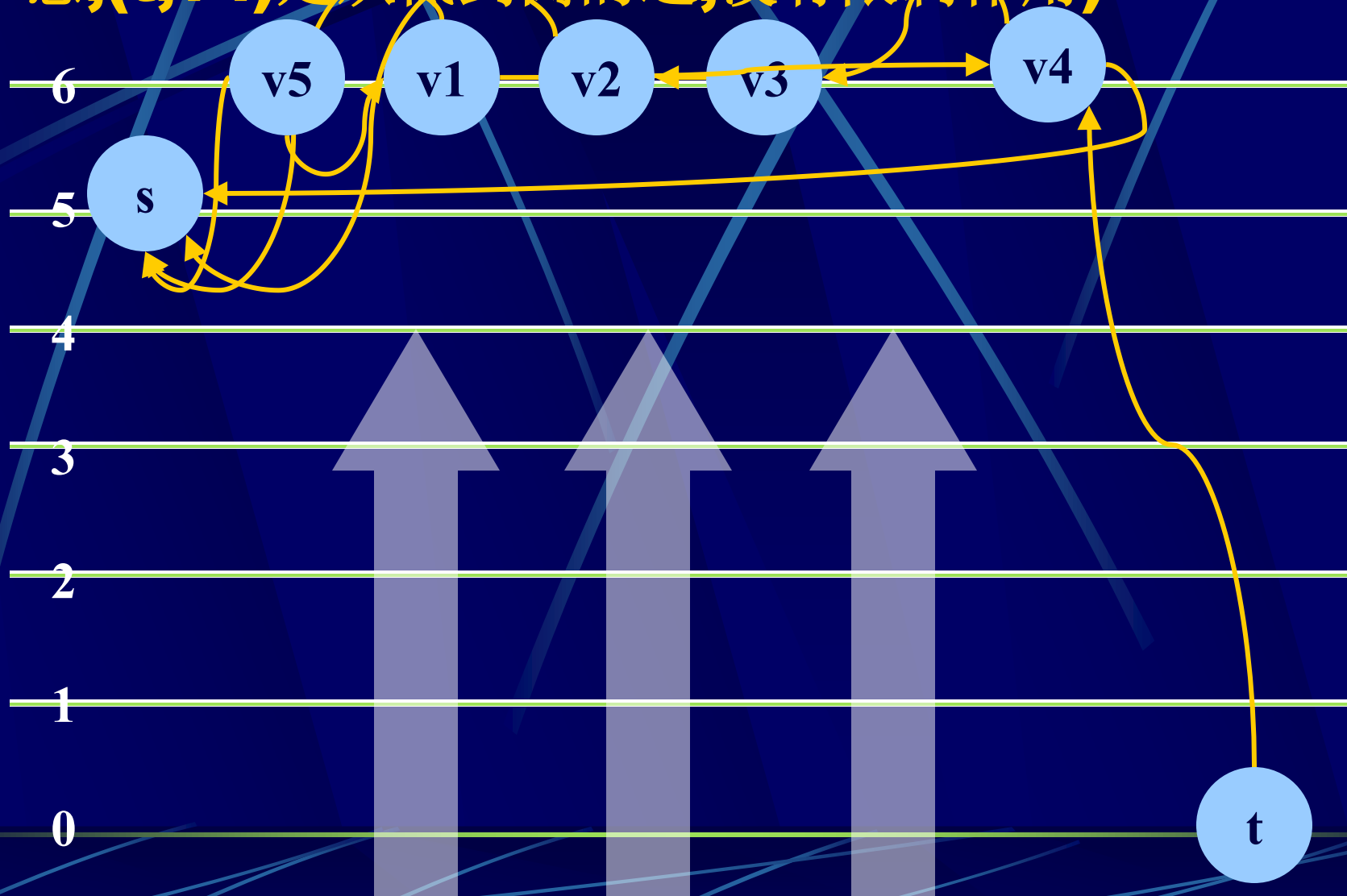
但是直接吹是吹不动的.残量网络中的每一条边都是一层限制(从高处指向低处的边才是限制.从低到高的边暂时是没有限制作用的),牢牢地将结点捆住.....至多把**v2**和**v3**吹到高度为**6**的位置上.



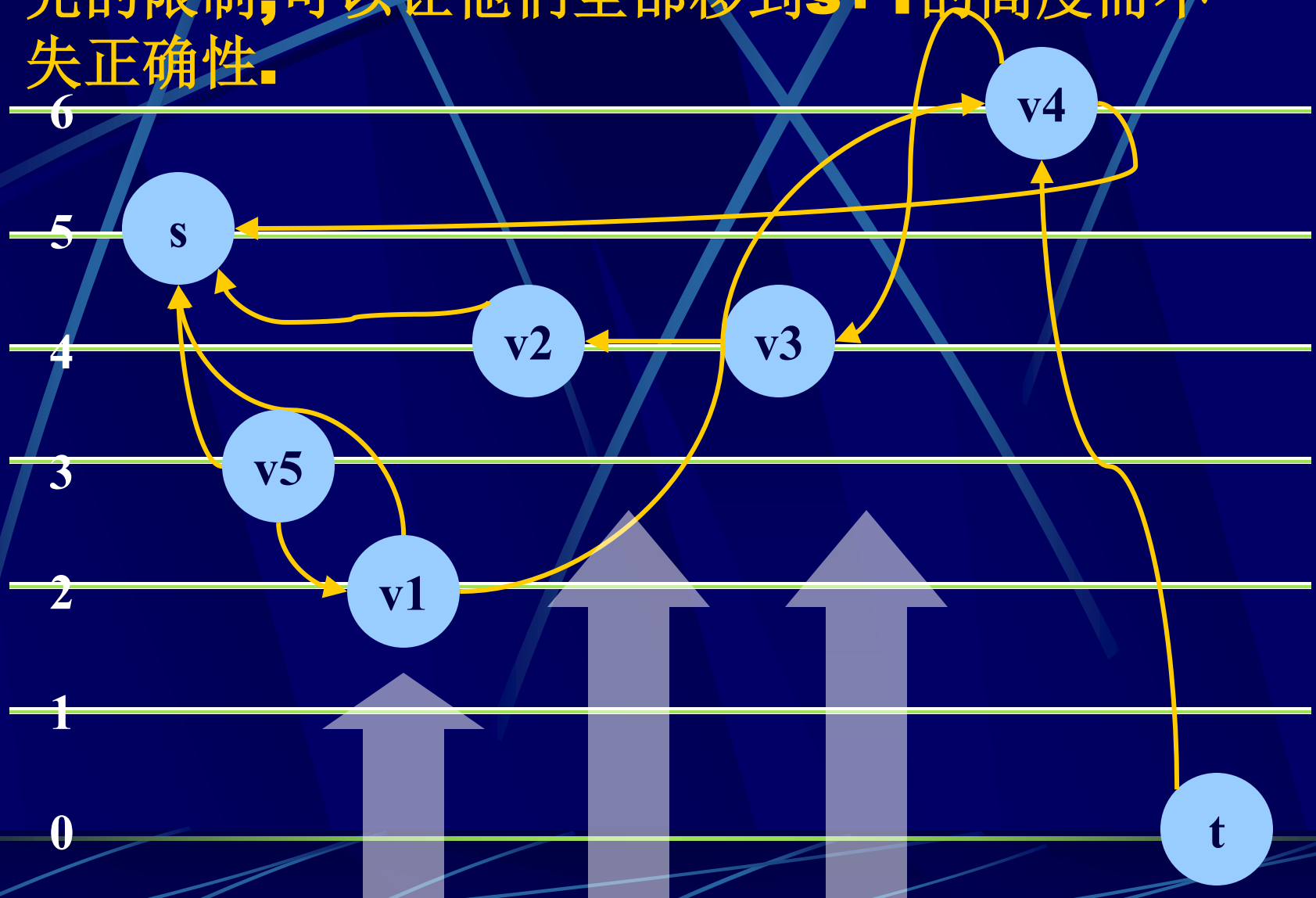
如果**v4**向**t**作了一次饱和推进,导致**(v4,t)**消失,这时我们发现,**v1,v5,v2,v3,v4**都几乎属于完全自由的状态!



v1, v5, v2, v3, v4都可以上升到高度**6**,而不影响**h**的合法性和递增性.重标号操作被大大减少.(注意,**(t, v4)**是从低到高的边,没有限制作用)



间隙优化:如果 $(0,s)$ 之间的某个高度 l 不存在任何结点,那么处在 (l,s) 之间的所有结点都失去了原先的限制,可以让他们全部移到 $s+1$ 的高度而不失正确性。



间隙优化

- 间隙优化十分有用。在具体实现时有惊人的效果。
- 正好，在**Highest-relabel**算法中，我们按结点的高度分成了 $2n$ 个块，只要顺便观察一下，如果有某个块为空了，那么立即应用间隙优化即可。
- 在某次**Check**操作结束后查看被检查的结点原先所在的块是否为空。
- 备注中为我的**Highest-relabel**算法的代码(带**BFS**预处理和间隙优化) (150行)

效率测试(时间单位:s)(忽略了 读数据的时间)

n	m	未加优化	加了两个优化
500	3000	0.60	0.06
500	30000	1.32	0.06
500	100000	1.43	0.06
800	100000	Too long	0.06 (根本测不出来.....)